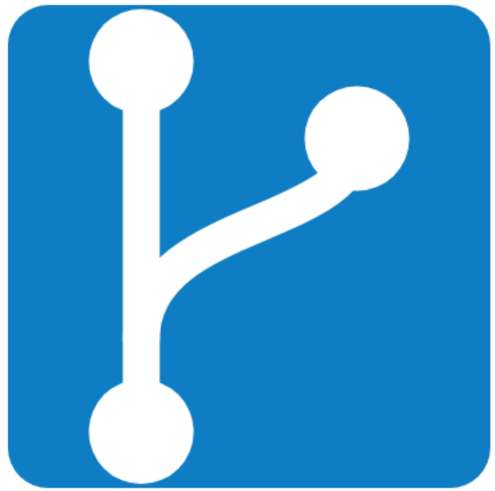
GIT & GITHUB

{ versionare il codice }



© 2026 Davide Fella | CC BY-NC 4.0
Uso didattico consentito - V 1.0

INDICE

01 Perché Git

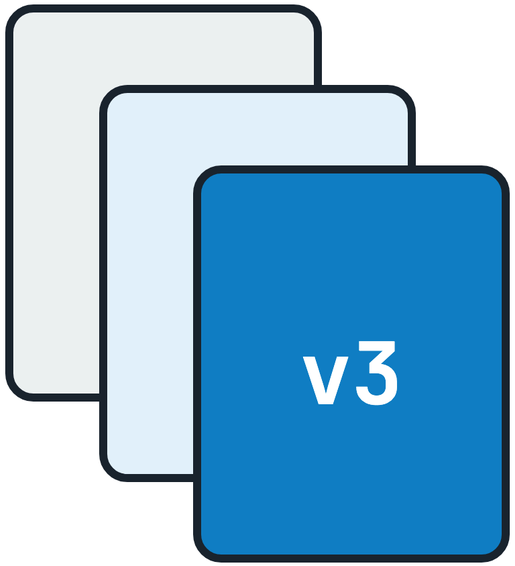
02 Come funziona

03 Git da terminale

04 GitHub

05 I branch

01
PERCHÉ GIT



IL PROBLEMA

Senza uno strumento, le versioni di un progetto diventano copie della cartella: quale è l'ultima? Che cosa è cambiato, quando, e perché?

Terminale

```
$ ls -1
battaglia
battaglia_finale
battaglia_finale_DEFINITIVA
battaglia_finale_DEFINITIVA (1)
battaglia_v2
```

IL CONTROLLO DI VERSIONE

Un **sistema di controllo di versione** (VCS) registra nel tempo le modifiche ai file di un progetto

- ***Cronologia:*** *ogni versione resta salvata, con autore, data e motivo*
- ***Ripristino:*** *si torna a qualsiasi versione precedente*
- ***Collaborazione:*** *più persone lavorano sullo stesso progetto senza sovrascriversi*
- ***Backup:*** *una copia completa del progetto vive anche su un server*

CENTRALIZZATO E DISTRIBUITO

Nei sistemi centralizzati la storia sta su un solo server. In Git ogni copia del progetto è un repository completo, con tutta la storia

	Centralizzato (SVN)	**Distribuito (Git)**
Storia	solo sul server	su ogni computer
Senza rete	non si salva	si lavora e si salva
Velocità	ogni operazione passa dalla rete	quasi tutto è locale
Se il server si rompe	**la storia è persa**	**ogni copia la contiene**

LINUS TORVALDS

Nel 1991 Linus Torvalds, studente a Helsinki, scrive il kernel Linux. Nel 2005 scrive anche Git, per gestirne il codice

- ***Linux:*** *il cuore di Android, della maggior parte dei server e di tutti i 500 supercomputer più potenti*
- ***BitKeeper:*** *dal 2002 il kernel usa questo sistema proprietario, gratuito. Nel 2005 la licenza gratuita viene revocata*
- ***Git:*** *il primo commit è del 7 aprile 2005, e pochi giorni dopo Git gestisce già il proprio codice*



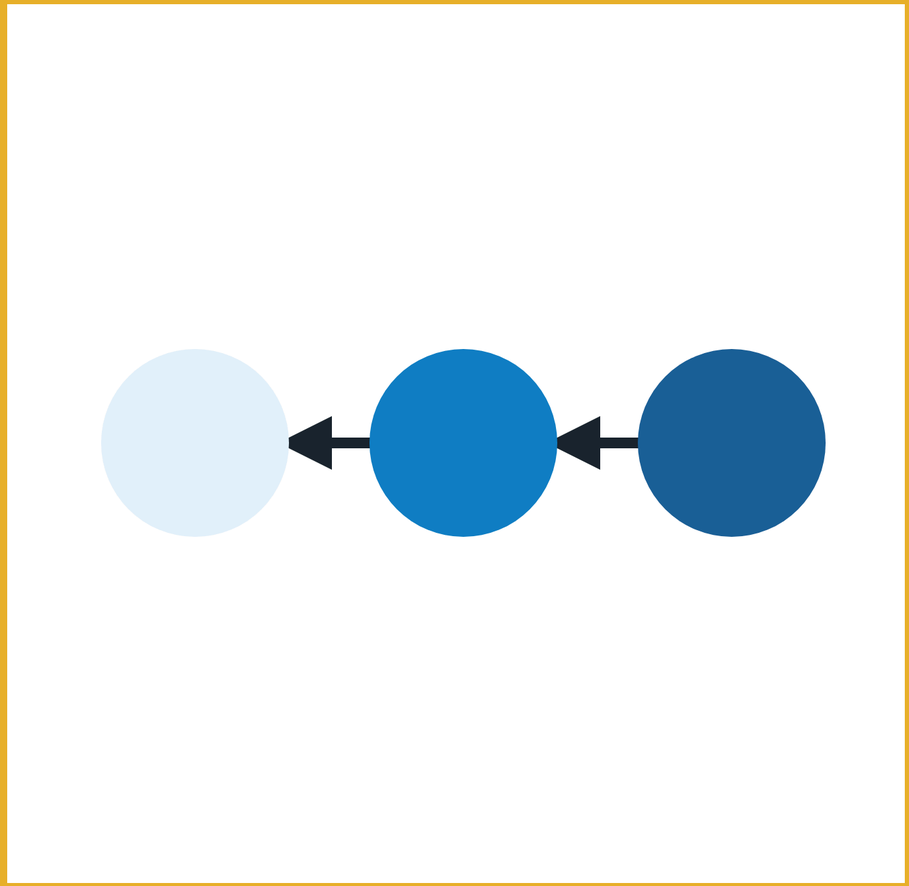
Linus Torvalds nel 2014. Foto di Krd, CC BY-SA 4.0

GIT E GITHUB

Non sono la stessa cosa: Git è il programma che gestisce la storia, GitHub è un servizio che ospita i repository Git

- ***Git:*** *si installa sul computer e funziona anche offline, senza nessun sito*
- ***GitHub:*** *sito web nato nel 2008, oggi di Microsoft: repository online, collaborazione, portfolio*
- ***Alternative:*** *GitLab e Bitbucket ospitano repository Git allo stesso modo*

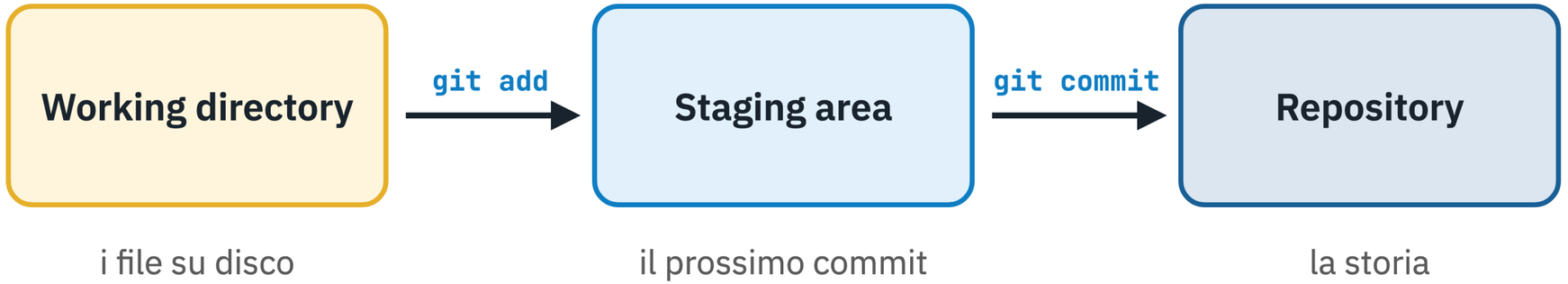
02
COME FUNZIONA



LE TRE AREE

Salvare una versione richiede due passi: si sceglie che cosa salvare con `git add`, poi lo si salva con `git commit`

- ***Working directory:*** *la cartella del progetto, con i file su cui lavori*
- ***Staging area:*** *le modifiche scelte per il prossimo commit*
- ***Repository:*** *la cartella nascosta* `.git`*, con tutta la storia*



GLI STATI DI UN FILE

Per Git ogni file è in uno di quattro stati, e `git status` dice in quale

Stato	Significato
untracked	file nuovo, che Git non ha mai salvato
modified	file cambiato rispetto all'ultimo commit
staged	modifica aggiunta con `git add`, entrerà nel prossimo commit
committed	**salvato nella storia, nessuna modifica in sospeso**

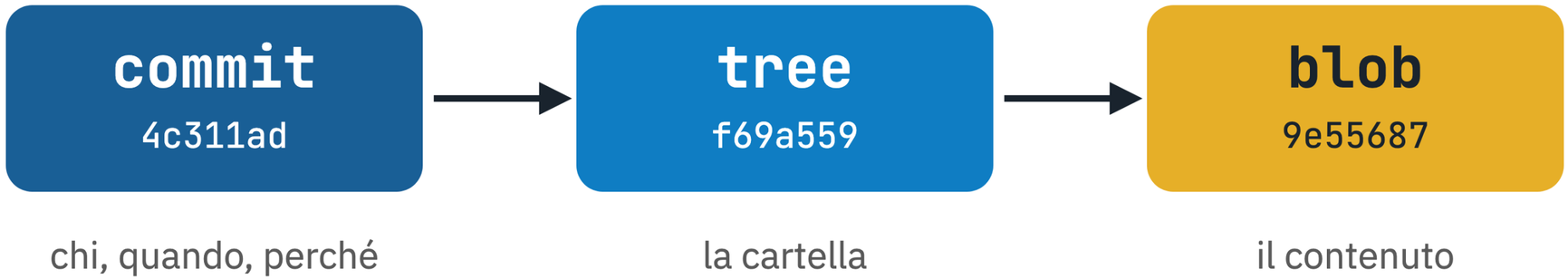
IL COMMIT È UNA FOTO

Un commit non salva le differenze: salva lo stato di **tutti** i file in quel momento, uno **snapshot**. Un file che non cambia non viene copiato, il nuovo commit riusa lo stesso contenuto



COM'È FATTO UN COMMIT

Dentro `.git` ci sono tre tipi di oggetti: il **commit** punta a un **tree**, l'elenco dei file della cartella, che punta ai **blob**, il contenuto di ogni file



DENTRO UN COMMIT

`git cat-file -p` mostra un commit così com'è salvato: un piccolo testo

- `tree`*: la cartella*
- `author`*: chi e quando*
- `parent`*: il commit precedente*
- ***Messaggio:*** *perché*

Terminale

```
$ git cat-file -p HEAD
tree f69a5591e003f4a02fed6be19367470d9c2f25fe
parent bb2bc17b6db737a935996b279352487ad765db35
author Mario Rossi <mariorossi@users.noreply.github.com> 1792430589 +0200
committer Mario Rossi <mariorossi@users.noreply.github.com> 1792430589 +0200

feat: messaggio di benvenuto
```

L'HASH

Ogni oggetto si chiama con 40 caratteri esadecimali, l'**hash** SHA-1, calcolato dal suo contenuto: cambia una lettera e cambia tutto l'hash

- ***Identità:*** *lo stesso contenuto ha sempre lo stesso hash*
- ***Integrità:*** *una storia modificata di nascosto non torna con gli hash*
- ***Abbreviato:*** *di solito bastano i primi 7 caratteri*

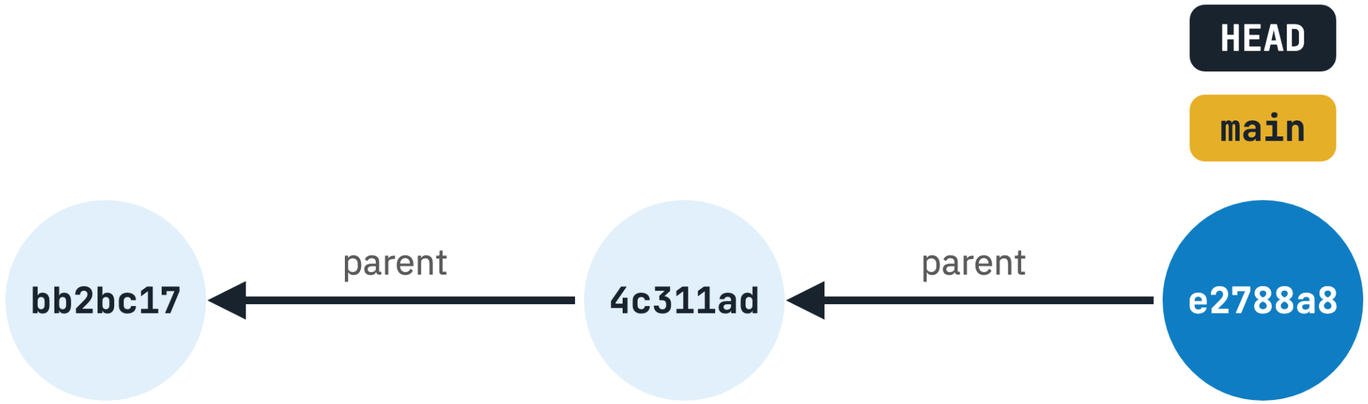
Terminale

```
$ echo "ciao" | git hash-object --stdin
887ae9333d92a1d72400c210546e28baa1050e44
$ echo "Ciao" | git hash-object --stdin
80493a57a211ecad9441216f4a2f28460b45c048
```

LA CATENA DI COMMIT

Ogni commit contiene l'hash del precedente, il **parent**. La storia è una catena che si percorre all'indietro, dal commit più recente fino al primo, che non ha parent

- ***Storia non modificabile:*** *cambiare un commit vecchio cambia il suo hash, e quindi quello di tutti i commit successivi*



MAIN E HEAD

Un **branch** è solo un nome che punta a un commit. `HEAD` indica il branch su cui stai lavorando

- `main`: *il branch principale, avanza da solo a ogni commit*
- `HEAD`: *dove sei adesso: il prossimo commit avrà come parent il commit di* `HEAD`

Terminale

```
$ cat .git/HEAD
ref: refs/heads/main
$ cat .git/refs/heads/main
4c311ad2e35c4518ff5ceed9df69c7670fbda738
```

03
GIT DA TERMINALE



LA CONFIGURAZIONE

Si fa una volta sola per computer. Nome ed email finiscono in ogni commit: l'email **noreply** di GitHub, in Settings → Emails, evita di pubblicare quella personale

```
git --version
git config --global user.name "Mario Rossi"
git config --global user.email "mariorossi@users.noreply.github.com"
git config --global init.defaultBranch main   # come GitHub
git config --global pull.rebase false         # git pull unisce con un merge
git config --list                             # controlla
```

GIT INIT

`git init` trasforma la cartella in un repository: crea `.git`, dove Git conserva oggetti, branch e configurazione. Cancellare `.git` significa cancellare la storia

Terminale

```
$ mkdir saluto
$ cd saluto
$ git init
Initialized empty Git repository in /Users/mario/saluto/.git/
```

GIT STATUS

`git status` è il comando da lanciare prima e dopo tutti gli altri: dice lo stato dei file e suggerisce il passo successivo

Terminale

```
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        saluto.js

nothing added to commit but untracked files present (use "git add" to track)
```

GIT ADD

`git add` mette le modifiche nella staging area. `git add .` aggiunge tutti i file della cartella

Terminale

```
$ git add saluto.js
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   saluto.js
```

GIT COMMIT

`git commit` salva la staging area nella storia. Il messaggio dice che cosa cambia e perché, in poche parole

- ***Prefisso:*** `feat:` *funzionalità,* `fix:` *correzione,* `docs:` *documentazione,* `chore:` *manutenzione*

Terminale

```
$ git commit -m "feat: primo saluto"
[main (root-commit) bb2bc17] feat: primo saluto
 1 file changed, 1 insertion(+)
 create mode 100644 saluto.js
```

GIT DIFF

`git diff` mostra le modifiche non ancora in staging, `git diff --staged` quelle che entreranno nel prossimo commit. Le righe con `+` sono aggiunte, quelle con `-` tolte

Terminale

```
$ git diff
diff --git a/saluto.js b/saluto.js
index 0ad3a4f..9e55687 100644
--- a/saluto.js
+++ b/saluto.js
@@ -1 +1,2 @@
 console.log("Ciao!");
+console.log("Benvenuti nel corso");
```

GIT LOG

`git log` percorre la catena dal commit più recente al primo. `--oneline` mostra una riga per commit, con l'hash abbreviato

Terminale

```
$ git log
commit 4c311ad2e35c4518ff5ceed9df69c7670fbda738
Author: Mario Rossi <mariorossi@users.noreply.github.com>
Date:   Mon Oct 19 19:23:09 2026 +0200

    feat: messaggio di benvenuto

commit bb2bc17b6db737a935996b279352487ad765db35
...
$ git log --oneline
4c311ad feat: messaggio di benvenuto
bb2bc17 feat: primo saluto
```

.GITIGNORE

Alcuni file non vanno nella storia: dipendenze come `node_modules`, file generati, password. Si elencano nel file `.gitignore`, che invece si committa

```
# .gitignore
node_modules/
.env
```

Terminale

```
$ git status
On branch main

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        package.json
```

TORNARE INDIETRO

Ogni errore ha il suo comando. `git status` li suggerisce quasi tutti

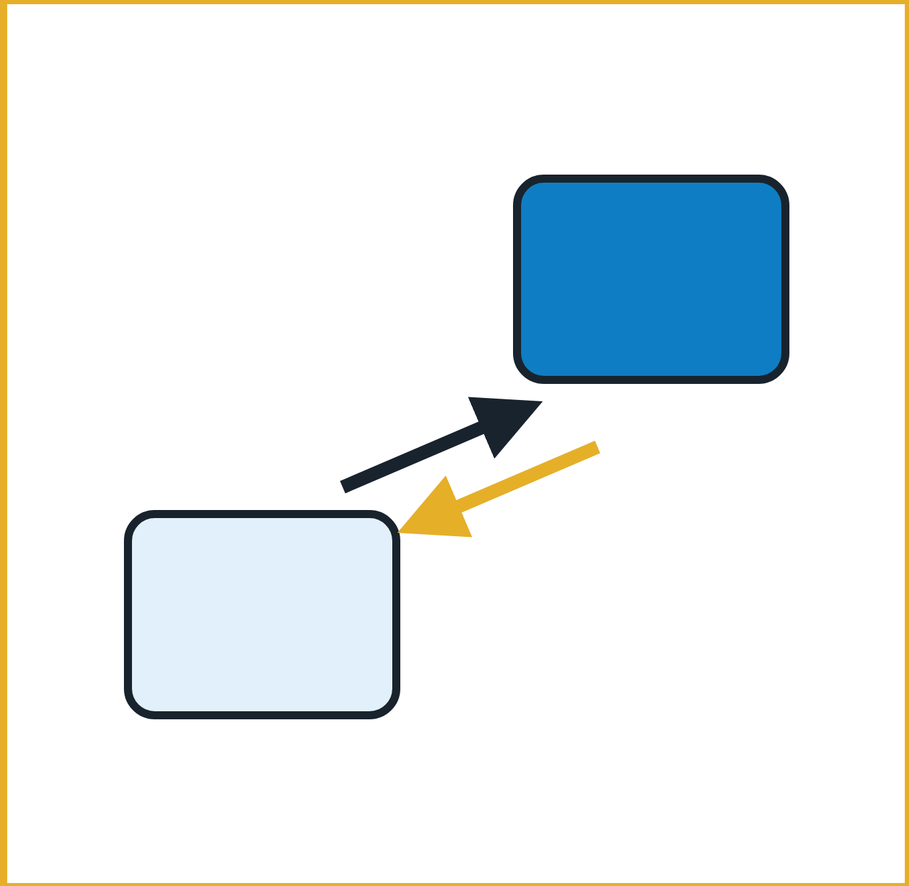
Situazione	Comando	Effetto
modifiche da buttare	`git restore file`	il file torna com'era
file aggiunto per sbaglio	`git restore --staged file`	esce dalla staging area
messaggio sbagliato	`git commit --amend`	riscrive l'ultimo commit, solo prima del push
commit da annullare	`git revert <hash>`	**nuovo commit che fa l'opposto**

RIEPILOGO

Il lavoro in locale si ripete sempre uguale: modifica, `git status`, `git add`, `git commit`

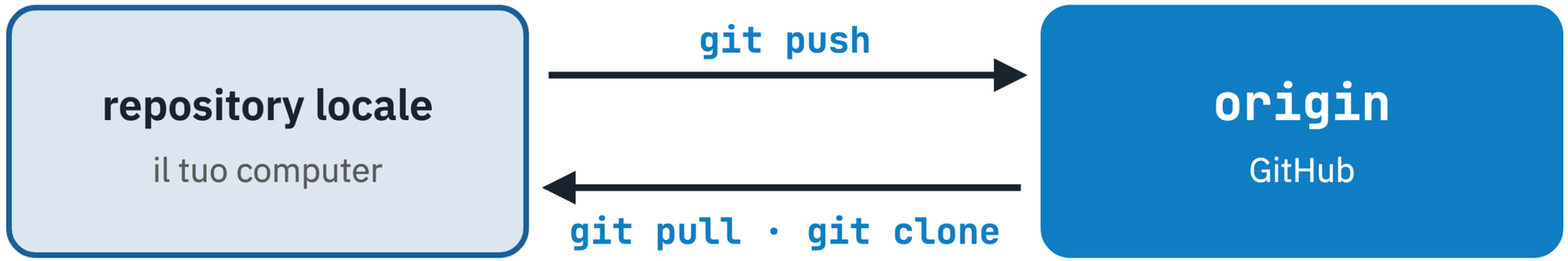
Comando	Che cosa fa
`git init`	crea il repository nella cartella
`git status`	stato dei file
`git add file`	modifica nella staging area
`git commit -m "…"`	salva la staging area nella storia
`git diff`	che cosa è cambiato
`git log --oneline`	la storia, una riga per commit

04
GITHUB



IL REPOSITORY REMOTO

Un **remote** è una copia del repository su un server, come GitHub. Di solito si chiama `origin`, e i commit passano da una copia all'altra solo quando lo chiedi



ACCEDERE A GITHUB

Dal 2021 GitHub non accetta la password dell'account per Git da terminale: serve un accesso dal browser o un token

- ***Windows:*** *Git Credential Manager, incluso in Git for Windows, apre il browser al primo push*
- ***macOS e Linux:*** *la GitHub CLI con* `gh auth login`*, poi Git usa le credenziali salvate*
- ***Token:*** *in alternativa, un personal access token al posto della password*

PUBBLICARE UN PROGETTO

Si crea su GitHub un repository vuoto, si collega come `origin` e si fa il primo push. `-u` ricorda il collegamento: dopo basta `git push`

Terminale

```
$ git remote add origin https://github.com/mariorossi/saluto.git
$ git push -u origin main
To https://github.com/mariorossi/saluto.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

GIT CLONE

`git clone` scarica un repository con tutta la storia e collega già `origin`.

Si usa per iniziare a lavorare su un progetto che esiste già

Terminale

```
$ git clone https://github.com/mariorossi/saluto.git
Cloning into 'saluto'...
$ cd saluto
$ git remote -v
origin  https://github.com/mariorossi/saluto.git (fetch)
origin  https://github.com/mariorossi/saluto.git (push)
```

PUSH E PULL

`git push` invia i commit locali, `git pull` scarica quelli nuovi e li unisce ai tuoi. Se su GitHub ci sono commit che non hai, il push viene rifiutato: prima `git pull`, poi di nuovo `git push`

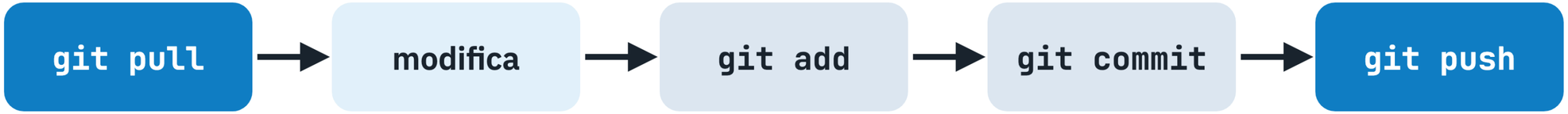
Terminale

```
$ git push
To https://github.com/mariorossi/saluto.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/mariorossi/saluto.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
```

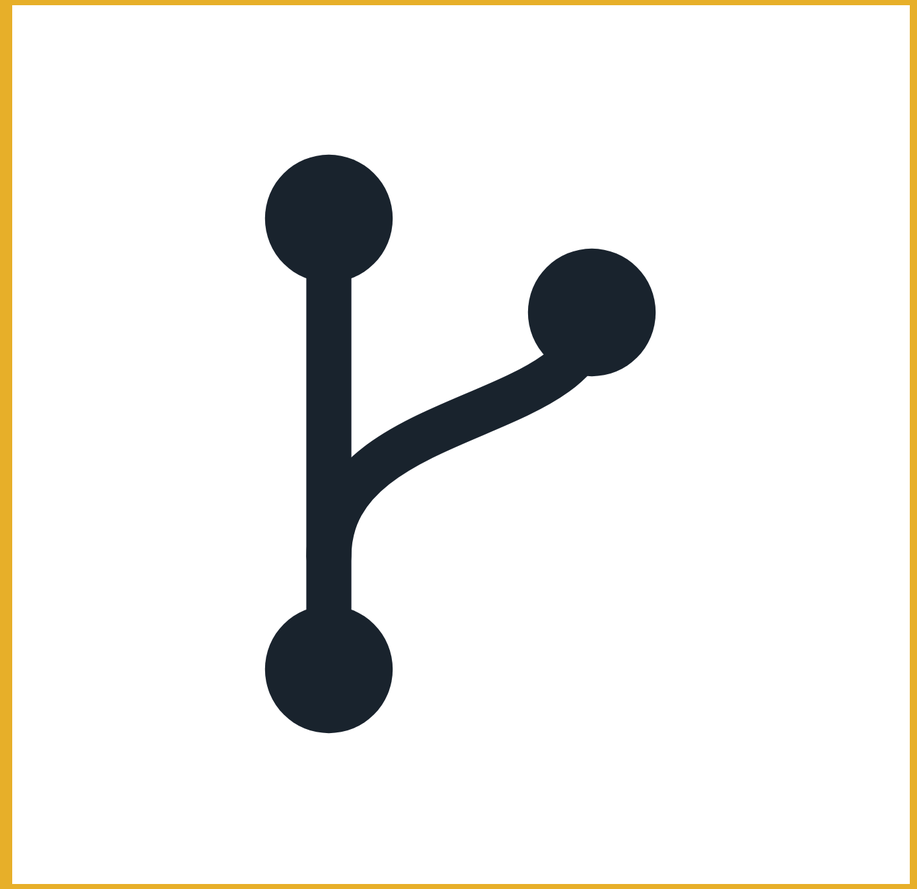
IL FLUSSO DI LAVORO

Con GitHub il ciclo si apre e si chiude con la rete: si parte da `git pull` per avere l'ultima versione, si chiude con `git push` per pubblicare la propria

- ***Commit piccoli:*** *uno per ogni passo che funziona, con un messaggio chiaro*
- ***Push frequenti:*** *quello che non è su GitHub esiste solo sul tuo computer*



05
I BRANCH



CREARE UN BRANCH

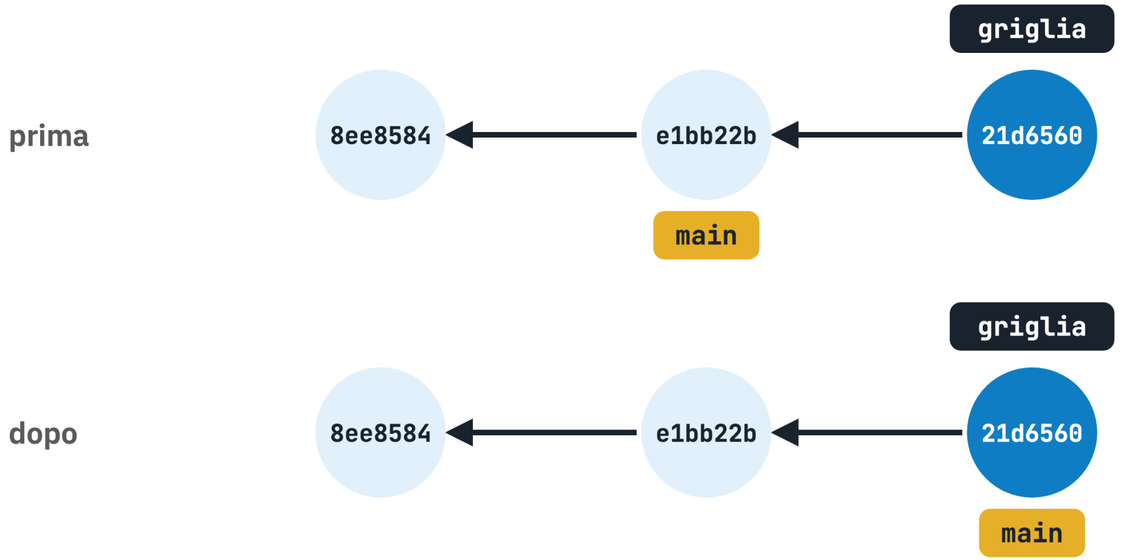
Un branch è un file con l'hash di un commit: crearlo costa niente. Serve a sviluppare una funzionalità senza toccare `main` finché non è pronta

Terminale

```
$ git switch -c griglia
Switched to a new branch 'griglia'
$ git add griglia.js
$ git commit -m "feat: stampa la griglia"
[griglia 21d6560] feat: stampa la griglia
 1 file changed, 1 insertion(+)
 create mode 100644 griglia.js
$ git switch main
Switched to branch 'main'
```

GIT MERGE

`git merge` porta in `main` i commit di un altro branch. Se `main` non è andato avanti, Git sposta solo il puntatore: è un **fast-forward**



Terminale

```
$ git merge griglia
Updating e1bb22b..21d6560
Fast-forward
 griglia.js | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 griglia.js
```

ERRORI COMUNI

I messaggi di Git sono lunghi, ma la soluzione di solito è scritta dentro

Messaggio	Causa e soluzione
`fatal: not a git repository`	sei nella cartella sbagliata, o manca `git init`
`Author identity unknown`	manca la configurazione di nome ed email
`nothing added to commit`	prima del commit serve `git add`
`! [rejected] (fetch first)`	su GitHub ci sono commit nuovi: `git pull`, poi `git push`
`Support for password authentication was removed`	serve l'accesso dal browser o un token

DOMANDE?



© 2026 Davide Fella | CC BY-NC 4.0
Uso didattico consentito - V 1.0